UVM Projects

Project Author: Arappagari Swethanjali
Institute: SURE proED

Project-1: Smart Four-Way Traffic Controller Verification using UVM

Project Description:

Developed a Smart Four-Way Traffic Controller using SystemVerilog and verified it using UVM. The controller manages traffic signals for the main road and side road using a Finite State Machine (FSM). It uses a vehicle detection sensor to control traffic flow and automatically switches between Green, Yellow, and Red signals based on predefined timings. Developed a UVM testbench to generate random sensor inputs and verify the traffic signal outputs.

Key Features:

- FSM-based four-way traffic controller.
- Four states: Main Green, Main Yellow, Side Green, and Side Yellow.
- Main-road and side-road traffic signal management.
- Sensor-based vehicle detection.
- Parameterized green and yellow signal durations.
- Constrained-random input generation.
- UVM Driver, Monitor, Sequencer, Agent, Scoreboard, Environment, and Test.
- Independent reference FSM for output checking.
- Safety checking to ensure both roads are not non-red simultaneously.
- PASS/FAIL reporting.

Tools Used: SystemVerilog, UVM, EDA Playground.

Verification: Verified the traffic controller using 300 constrained-random transactions. Used a UVM monitor and scoreboard to compare expected and actual traffic signals and check traffic safety conditions.

EDA Link: https://www.edaplayground.com/x/igZL

Project-2: 8b/10b Encoder and Decoder Verification using UVM

Project Description:

Developed an 8b/10b Encoder and Decoder using SystemVerilog and verified them using UVM. The encoder converts 8-bit data into a 10-bit encoded output, while the decoder converts it back into the original 8-bit data. The design supports special K-control characters and running disparity management. Developed a UVM testbench with directed and constrained-random sequences to verify encoding and decoding operations.

Key Features:

- 8-bit to 10-bit encoding and 10-bit to 8-bit decoding.
- Supports data and K-control characters.
- Implements 5b/6b and 3b/4b encoding.
- Running disparity management (RD+ and RD−).
- Detects invalid codewords and disparity errors.
- Directed testing of all 256 data bytes and 12 legal K-characters.
- Generates 1,000 constrained-random transactions.
- Supports random idle cycles.
- Uses UVM Driver, Monitor, Sequencer, Agent, Scoreboard, Environment, and Test.

- Functional coverage and PASS/FAIL reporting.

Tools Used: SystemVerilog, UVM, EDA Playground.

Verification: Verified the encoder and decoder using directed and constrained-random test cases. The scoreboard checks encoded data rules, running disparity, K-character encoding, decoded data, and error signals. Functional coverage is used to track data combinations and K-character selection.

EDA Link: https://www.edaplayground.com/x/uTei

Project-3: Smart Vending Machine Controller Verification using UVM

Project Description:

Developed a Smart Vending Machine Controller using SystemVerilog and verified it using UVM. The vending machine accepts nickel (5 cents) and dime (10 cents) coins. When the inserted amount reaches the product price of 15 cents, the machine dispenses the product. If the inserted amount exceeds 15 cents, it also returns 5 cents as change. A UVM testbench was developed to verify coin combinations, dispensing, and change-return operations.

Key Features:

- FSM-based vending machine controller with three states: S0, S5, and S10.
- Accepts nickel (5 cents) and dime (10 cents) coins.
- Product price set to 15 cents.
- Generates a dispense signal when the required amount is reached.
- Returns 5 cents change when 20 cents is inserted.
- Registered outputs for dispense and change.

- Supports idle cycles without losing the inserted amount.
- UVM components include Transaction, Directed Sequence, Random Sequence, Driver, Monitor, Sequencer, Agent, Environment, Scoreboard, and Test.
- Scoreboard uses an independent arithmetic reference model.
- Functional coverage for inserted amount and coin combinations.
- PASS/FAIL reporting.

Tools Used: SystemVerilog, UVM, EDA Playground.

Verification: Used directed test scenarios to check different coin combinations, including exact payment, excess payment, and idle cycles. Also generated 500 constrained-random transactions. The scoreboard compares the expected dispense and change outputs with the DUT outputs and reports functional coverage and PASS/FAIL counts.

EDA Link: https://www.edaplayground.com/x/Npuw

Project-4: Half Adder Design and Verification using UVM

Project Description:

Developed a combinational Half Adder using SystemVerilog and verified its functionality using UVM. The Half Adder takes two 1-bit inputs and generates two outputs: Sum and Carry. A UVM-based testbench was created to apply directed and constrained-random inputs and check the outputs using a scoreboard.

Key Features:

- Combinational Half Adder design.
- Sum calculated using XOR logic (a ^ b).

- Carry calculated using AND logic (a & b).
- UVM transaction class for input and output data.
- Directed sequence covering all four input combinations twice.
- Constrained-random sequence generating 100 transactions.
- UVM Driver, Monitor, Sequencer, Agent, Environment, and Scoreboard.
- Scoreboard compares DUT outputs with expected Sum and Carry.
- Functional coverage for input and output combinations.
- PASS/FAIL reporting.

Tools Used: SystemVerilog, UVM, EDA Playground.

Verification: Applied all four possible input combinations using directed testing, followed by 100 random transactions. The UVM monitor collects the inputs and outputs, while the scoreboard checks the Sum and Carry against the expected results. Functional coverage and PASS/FAIL counts are reported at the end of simulation.

EDA Link: https://www.edaplayground.com/x/naAv